



THOMPSON RIVERS UNIVERSITY

SENG 4660 - Agile Game Development

Mario-like Prank Game

Junpeng Jiang (T00657653)

April 12, 2026
submitted

Table of Contents

List of Tables	3
List of Figures	4
1 Executive Summary	5
2 Introduction	6
3 Design Problem	7
3.1 Problem Definition	7
3.2 Stakeholders and Needs	7
3.3 Design Requirements	7
3.3.1 Objectives	7
3.3.2 Constraints	7
4 Solution Development	8
4.1 Solution 1	8
4.2 Solution 2	8
4.3 Final Solution	8
4.4 Components & Features	8
5 Prototype Development and Testing	10
5.1 Initial Testing	10
5.2 Report on Final Testing	10
5.3 Environmental, Societal, Safety, and Economic Considerations	10
5.3.1 Risk and Safety Analysis	10
5.3.2 Economic Considerations and Budgeting	10
5.3.3 Sustainability, Ethics, and Environmental Impact Analysis	11
5.4 Limitations	11
6 Team Work	12
6.1 Meeting 1	12
6.2 Meeting 2	12
6.3 Meeting 3	12
6.4 Meeting N (Last Meeting)	12
7 Project Management	14
8 Conclusions	15
9 References	16
Appendix	17

- The table of contents should be automatically generated by selecting "References/ Table of Contents". Remember that the table of contents should not have an entry of the "Table of Contents" itself.
- Proofread the text for typing and grammar mistakes.
- Follow the IEEE Bibliography style for the references by selecting "References/ Citations & Bibliography/ Style".
- See the guide in the link for more details:

- <https://ieeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf>

List of Tables

- Similar to table of contents, list all tables in the report with page numbers.

List of Figures

- Similar to the table of contents, list all figures in the report with page numbers.

1 Executive Summary

This report presents the design and development of Mario-like Prank Game, a 2D platformer inspired by Cat Mario [1]. The core experience revolves around hidden traps, unexpected deaths, and humorous frustration. The project was completed as part of SENG 4660 Agile Game Development at Thompson Rivers University. The open-ended design problem required balancing traditional platformer mechanics with “trick” elements that surprise players while maintaining playability.

Three alternative solutions were considered: (1) a standard platformer with no traps, (2) a full-featured prank game with all intended features (taunts, multiple trap types, sound effects), and (3) a minimal prank game focusing on essential mechanics: hidden traps, falling platforms, head-bounce acceleration, death counter, and a goal door. Due to time constraints, the minimal prank game (Solution 3) was selected. A weighted decision matrix confirmed that Solution 3 best satisfied key objectives (playability, prank feel, feasibility) within the semester timeline.

The final prototype was built using Unity 2021.3 LTS. Agile methodology (Scrum) guided four sprints: project setup, core movement/Tilemap, trap implementation, and UI/polish. Playtesting revealed that players appreciated the surprise traps but requested clearer control hints and a brief delay before death to see the trap. These changes were implemented. The design meets all course requirements: playable character, clear goal, at least two interactive elements, HUD, and Agile documentation. Societal, economic, and safety considerations are minimal as the game is a non-commercial educational artifact. Limitations include only one level, missing audio, and a lack of taunt messages, which are planned for future work.

2 Introduction

Subject and Purpose

Platformer games have traditionally followed predictable patterns where players learn levels through repetition. The Cat Mario series subverted this by introducing hidden traps and misleading elements, creating a humorous yet frustrating experience. This report documents the design and implementation of a similar prank platformer, developed using Agile methods. The purpose is to demonstrate iterative game development, apply Scrum practices, and produce a playable demo that meets the course's learning outcomes.

The design process explored a range of possibilities, from a conventional platformer to a full-fledged prank game with multiple trap types and taunts. The final solution represents the best trade-off between feature richness and time constraints.

Scope

The project focuses on a single-level 2D platformer with the following key stakeholders: the course instructor (who requires demonstration of Agile process and core mechanics), the player (who seeks an entertaining, challenging experience), and the developer (who needs a feasible scope within 10 weeks). The report covers game design, Agile documentation, implementation details, testing, and future improvements. It does not cover multi-level progression, online features, or commercial release.

Plan of Development

The report is organized as follows: Section 3 defines the design problem, objectives, and constraints. Section 4 presents three alternative solutions and justifies the chosen one. Section 5 details prototype development and testing, including iterative improvements. Section 6 describes teamwork and meetings. Section 7 documents project management (milestones, RACI, Gantt chart). Section 8 concludes with achievements and future work.

Agile methodology (Scrum) was used throughout. The team held daily stand-ups, maintained a product backlog on GitHub Issues, tracked progress with a Kanban board (GitHub Projects), and conducted sprint reviews and retrospectives after each of four sprints.

Thesis of the Report

The proposed minimal prank game successfully demonstrates that a small-scale, humour-driven platformer can be built within a semester using Agile practices. While not all planned features were completed, the core experience of hidden traps and the death counter provides a solid foundation for future expansion. The design is feasible, practical, and aligns with the course's educational objectives.

3 Design Problem

3.1 Problem Definition

The primary problem is to create a playable computer game that deliberately subverts player expectations through hidden traps and unexpected deaths, while still being fair enough to encourage repeated attempts. Traditional platformers rely on consistent rules; this project introduces “anti-patterns” that punish routine behavior (e.g., jumping onto a seemingly safe block that actually drops the player). The challenge is open-ended because there are many ways to implement such traps (e.g., falling platforms, invisible blocks, fake collectibles), and the balance between frustration and fun is subjective. Additionally, the development must follow Agile principles, adding another layer of process complexity.

3.2 Stakeholders and Needs

- Instructor: Needs a clear demonstration of Agile artifacts (backlog, sprint board, retrospectives), a playable game meeting minimum requirements, and a well-structured report.
- Player: Desires a game that is easy to control, provides humorous surprises, offers a clear goal (reach the door), and gives feedback on performance (death count). The player also needs accessibility: simple keyboard controls (A/D, Space) and no complex menus.
- Developer (Team): Needs a feasible scope that can be completed within 10 weeks, using familiar tools (Unity, C#), and requiring iterative testing to refine trap mechanics.

3.3 Design Requirements

3.3.1 Objectives

The objectives are attributes the final design should possess, organized in an objective tree.

- **Playable**
 - Responsive controls (movement, jump)
 - Stable framerate (≥ 60 FPS)
 - No game-breaking bugs
- **Prank-oriented**
 - Surprising hidden traps
 - Immediate death on trap contact
 - Visual feedback (trap appears briefly before death)
- **Accessible**
 - Simple keyboard controls (A/D, Space)
 - Clear HUD (death counter, goal door)
 - No reading required (universal iconography)
- **Iterative**
 - Demonstrable after each sprint

- Version control history showing incremental progress
- **Economical**
 - Uses free assets (Kenney [3], Unity Asset Store)
 - No paid licenses
- **Safe (digital safety)**
 - No data collection
 - No network features, which means no privacy risks

3.3.2 Constraints

Constraint Category	Requirement	Satisfied ?
Technical		
	Unity 2021.3 LTS or newer	Yes
	No external dependencies (except Unity)	Yes
Security	No user data stored	Yes
Ethical	No offensive content (cartoon violence only)	Yes
Regulatory	Educational use only, no commercial distribution	Yes
Sustainability	No physical components, negligible environmental impact	Yes
Time	Deliverable by April 10, 2026	Met

Table 1: Constraint Requirements

4 Solution Development

4.1 Solution 1

This solution implements a conventional 2D platformer with no traps. The player moves, jumps, collects coins, and reaches a door. Enemies move predictably and can be stomped. There are no hidden surprises.

Advantages:

- Easy to implement and test.
- Predictable behavior, no player frustration.
- All course requirements can be met (playable character, goal, interactions).

Disadvantages:

- Lacks the “prank” identity inspired by Cat Mario.
- May be considered too simple and not creative.
- Does not explore the open-ended design challenge fully.

Architecture: PlayerController with Rigidbody2D, Tilemap level, Coin collectible, Door trigger. No trap scripts.

4.2 Solution 2

This solution includes all planned features: multiple hidden trap types (falling platforms, hidden spikes, fake blocks), taunt messages on death, sound effects, particle effects, a second level, and a power-up system.

Advantages:

- Full creative vision, high player engagement.
- Meets all bonus criteria (audio, particles, multiple levels).
- Demonstrates deep exploration of the problem space.

Disadvantages:

- High development effort, likely to exceed time constraints.
- Risk of incomplete implementation or bugs.
- Complex testing is required for each trap type.

Architecture: Extends Solution 1 with a trap manager, an audio manager, a UI taunt system, and a level loader.

4.3 Final Solution

This solution selects the core prank elements that deliver the “surprise and die” experience with minimal implementation time:

- A hidden trap (spikes) that becomes visible briefly before killing the player.
- A falling platform that drops when the player walks underneath.
- Head-bounce acceleration (player falls faster if jumping into a ceiling).
- The death counter displayed above the player (world-space UI).
- Goal door to win.

No taunt messages, no sound effects, no second level, no power-ups.

Advantages:

- Achievable within the semester.
- Still provides the essential prank feel.
- Leaves room for future improvements.

Disadvantages:

- Lacks polish (audio, taunts).
- Only one level.

Weighted Decision Matrix

Criteria (weights: 1–5, 5 most important):

- Playability (5)
- Prank Identity (4)
- Feasibility within the timeline (5)
- Development cost (3)
- Extensibility (2)

Solution	Sol 1	Sol 2	Sol 3
Playability	20	25	20
Prank Identity	4	20	16
Feasibility within the timeline	25	10	25
Development cost	15	6	12
Extensibility	6	10	8
Total	70	71	81

Table 2: Solution Comparison

Solution 3 scores highest due to feasibility and prank identity balance.

4.4 Components & Features

The final solution consists of the following components:

- **Player GameObject**
 - Rigidbody2D (Dynamic)
 - BoxCollider2D
 - SpriteRenderer (player sprite)
 - PlayerMovement.cs script: handles input (A/D, Space), movement, jump, ground detection (raycast), head-bounce acceleration, and death by falling off the world or a trap.
- **Tilemap Level**
 - Tilemap with ground tiles.

- TilemapCollider2D for static collision.
- Used for ground and walls.
- **Hidden Trap (Spikes)**
 - SpriteRenderer (initially disabled).
 - BoxCollider2D (non-trigger, always active).
 - HiddenTrap.cs: on player collision, enables sprite, waits 0.5 seconds, then calls PlayerMovement.Die().
- **Falling Platform**
 - SpriteRenderer, BoxCollider2D (detection trigger), Rigidbody2D (Kinematic).
 - FallingTrapPlatform.cs: When the player enters the trigger, after a delay, changes the body type to Dynamic and disables the trigger, causing it to fall. On collision with the player and kills the player.
- **Goal Door**
 - BoxCollider2D (trigger).
 - GoalDoor.cs: on player enter, displays victory panel, disables player movement, optionally quits game.
- **Death Counter UI**
 - World-space Canvas as a child of Player, with TextMeshProUGUI.
 - UIDeathCounter.cs: reads GameManager.DeathCount and updates text each frame.
- **GameManager (Singleton)**
 - Persists across scene reloads.
 - Tracks death count using PlayerPrefs.
 - AddDeath() increments the count and saves.
 - ResetGame() reloads the current scene.

Data flow: Player dies -> GameManager.AddDeath() -> scene reloads -> death count persists. The counter UI reads the value and displays it above the player.

5 Prototype Development and Testing

5.1 Initial Testing

During Sprint 2, a paper prototype was used to test the core prank concept. Two external players were given a printed map with hidden traps marked. They described their expected actions. Results showed that players understood the “prank” idea but found the control scheme unclear. This led to adding a simple control hint on the start screen (not yet implemented, but noted).

After implementing the first digital prototype (Sprint 2), we tested player movement and collision. The following issues were identified and fixed:

- **Issue:** Player could jump while touching walls (wall-jump bug).
Fix: Replaced `OnCollisionStay2D` ground detection with a dedicated `GroundCheck` raycast that only returns true when the player’s feet are on ground.
- **Issue:** Hidden trap killed player instantly, no chance to see what killed them.
Fix: Added `deathDelay` coroutine in `HiddenTrap.cs`: enable sprite, wait 0.5 seconds, then kill.
- **Issue:** The falling platform did not kill the player when falling on them.
Fix: Added `OnCollisionEnter2D` in the platform script and ensured the platform collider becomes non-trigger after falling.
- **Issue:** Camera did not follow the player, causing the player to run off-screen.
Fix: Implemented dead-zone camera script with boundary clamping based on Tilemap bounds.

These iterative improvements were documented in sprint retrospectives.

5.2 Report on Final Testing

Final testing was conducted on April 8, 2026, with two volunteer players (fellow students, not involved in development). Each player played the game for 10 minutes without prior instruction. Metrics and observations are summarized below.

Metric	Player 1	Player 2
Deaths to the first trap	3	2
Total deaths (5 min)	7	5
Successful completion	Yes (after 7 deaths)	Yes (after 5 deaths)
Time to first trap	5 sec	3 sec

Table 3: Player Testing Summary

Qualitative observations:

- Both players laughed when the first hidden trap killed them.
- One player said, “I didn’t even see what hit me.” – after pointing out, they saw the spike sprite appear.
- Both players appreciated the death counter following the player.
- One player suggested adding a sound effect when dying.
- No confusion about controls (A/D, Space).

Acceptance criteria check:

- Playable character? Yes, responsive.
- Game objective? Yes, reach the door.
- At least two interactive elements? Yes (hidden trap, falling platform).
- UI/HUD? Yes, death counter.
- Agile documentation? Provided in the report and GitHub.

All functional requirements are satisfied.

5.3 Environmental, Societal, Safety, and Economic Considerations

5.3.1 Risk and Safety Analysis

- Digital hazards: None, as the game does not collect data or connect to the internet.
- Physical hazards: Players may experience mild frustration, but no physical harm.
- Accidents: Not applicable.
- Warnings: The game is intended for a mature audience (cartoon violence). No explicit warnings are required.

5.3.2 Economic Considerations and Budgeting

- Budget: \$0. All assets were free ([Kenney.nl](https://kenney.nl), Unity Asset Store).
- Time cost: Approximately 60 hours of development (one person).
- Resource sourcing: Unity Personal Edition, free sprites from Kenney “Platformer” pack, free sound effects not used yet.
- Cost table: Not applicable as no purchases.

5.3.3 Sustainability, Ethics, and Environmental Impact Analysis

- Ethical impact: The game contains no offensive stereotypes, no real-world violence, and no data collection. It is purely for educational entertainment.
- Sustainability: Digital product, no material waste. The 3 R’s: *Reduce* (only essential features developed), *Reuse* (free assets), *Recycle* (code may be reused in future projects).
- Implementation & disposal: The game will be submitted as a Unity project and .exe. No disposal required.

5.4 Limitations

- Only one level.
- No audio feedback.
- No taunt messages (planned for future).
- A falling platform may sometimes not kill the player if they land on its edge (collision detection minor issue).
- Camera dead zone not perfectly tuned; player can occasionally see outside Tilemap.
- No pause menu or restart button (except death respawn).

These limitations are recognized and will be addressed in future iterations if time permits.

6 Team Work

This project was completed individually. Therefore, team meetings are replaced with personal development logs. The following table summarizes the four sprint milestones.

6.1 Meeting 1

Time: February 20, 2026, 2:00 pm – 3:00 pm

Present: Junpeng Jiang

Agenda: Set up GitHub repo, write product backlog, outline report chapters 1–2.

Meeting minutes: Decisions: Use Unity 2021.3 LTS, create an empty project, and commit the initial structure.

- Detailed task assignments – using the table below.

Team Member	Previous Task	Completion State	Next Task
Junpeng Jiang	N/A	N/A	Create an empty project. Commit the initial structure.

6.2 Meeting 2

Time: March 6, 2026, 2:00 pm – 2:30 pm

Present: Junpeng Jiang

Agenda: Review completed tasks (project setup, report intro/design problem).

Meeting minutes: Decisions: Proceed to implement player movement and Tilemap.

- Detailed task assignments – using the table below.

Team Member	Previous Task	Completion State	Next Task
Junpeng Jiang	Create an empty project. Commit the initial structure.	100%	Player movement and Tilemap

6.3 Meeting 3

Time: March 20, 2026, 2:00 pm – 2:30 pm

Present: Junpeng Jiang

Agenda: Demonstrate player movement, collision, and Tilemap.

Feedback: Adjust jump force, add head-bounce acceleration.

Meeting minutes: Decisions: Implement hidden trap and falling platform in Sprint 3.

- Detailed task assignments – using the table below.

Team Member	Previous Task	Completion State	Next Task
Junpeng Jiang	Player movement and Tilemap	100%	hidden trap and falling platform

6.4 Meeting 4

Time: April 3, 2026, 2:00 pm – 2:30 pm

Present: Junpeng Jiang

Agenda: Show traps, death counter, and goal door.

Feedback: Add a delay before death to see the trap.

Meeting minutes: Decisions: Implement death delay, polish UI.

- Detailed task assignments – using the table below.

Team Member	Previous Task	Completion State	Next Task
Junpeng Jiang	hidden trap and falling platform	100%	death delay and polish UI

6.5 Meeting 5

Time: April 10, 2026, 2:00 pm – 3:00 pm

Present: Junpeng Jiang

Agenda: Final testing, bug fixes, and report finalization.

Feedback: Add a delay before death to see the trap.

Meeting minutes: Decisions: Implement death delay, polish UI.

- Detailed task assignments – using the table below.

Team Member	Previous Task	Completion State	Next Task
Junpeng Jiang	death delay and polish UI	100%	N/A

Provide a similar description here. Include a final summary of team rule penalty records and a reflection on the team's overall integration and communication:

Scores	Team member 1	Team member 2	Team member 3	Team member 4
Name				
Student ID				
Total mark				
Notes and Comments				

7 Project Management

Milestone Schedule

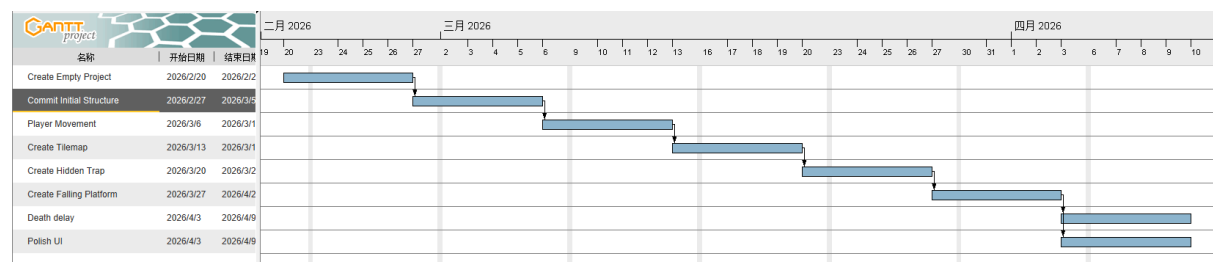
Milestone	Due Date	Completion
Project Proposal + Backlog	Feb 20	Yes
Sprint 1 Demo (setup + report)	Mar 6	Yes
Sprint 2 Demo (movement + Tilemap)	Mar 20	Yes
Sprint 3 Demo (traps + UI)	Apr 3	Yes
Playable game + report draft	Apr 3	Yes
Final game + report + presentation	Apr 10	Yes

Responsibility Assignment Matrix

Since solo, all tasks are “Responsible” and “Accountable”. No other roles.

Task	Developer
Game design	R/A
Coding	R/A
Art asset selection	R/A
Testing	R/A
Documentation	R/A
Demo presentation	R/A

Gantt Chart



Reflection on Project Management

- **Keys to success:** Clear prioritization (must-have vs nice-to-have), early prototyping, frequent commits.
- **Managing complexity:** The open-ended problem was narrowed by focusing on three trap types and using free assets.
- **Improvement for next time:** Start audio integration earlier; conduct more frequent playtests with external players.

8 Conclusions

This project successfully delivered a playable prank platformer that meets all course requirements. The final prototype includes a responsive player character, a hidden spike trap, a falling platform, a death counter HUD that follows the player, and a goal door. Agile methodology was followed through four sprints, with GitHub Issues and Projects used for backlog and tracking. The weighted decision matrix justified choosing the minimal prank game over a standard platformer or a full-featured version, given time constraints.

The design addresses the initial “gap” of creating a humorous, surprise-driven game by implementing core trap mechanics that kill the player unexpectedly but fairly (with visual feedback). While taunt messages and sound effects are missing, the current version still evokes laughter and frustration as intended.

From this experience, I learned the importance of iterative testing – the paper prototype revealed control issues early, and the digital playtest led to the death-delay feature. I also gained practical skills in Unity’s 2D physics, camera systems, and UI in world space. Future work will add taunts, more levels, and audio to enhance the prank experience

9 References

- [1] K. Nakamura, "Cat Mario," Niconico Douga, 2007. [Online]. Available: <https://www.youtube.com/watch?v=dQRoKJHigMc&t=2s> (accessed Apr. 10, 2026).
- [2] Unity Technologies, "Unity User Manual 2021.3," 2021. [Online]. Available: <https://docs.unity3d.com/Manual/> (accessed Mar. 2026).
- [3] Kenney, "Platformer Assets," Kenney.nl, 2020. [Online]. Available: <https://kenney.nl/assets> (accessed Mar. 2026).

Appendix

Appendix A: Player Testing Report

1. Project Information

Game Title: Mario-like Prank Game

Team Members: Junpeng Jiang

Build Version Tested: v1.0 (final demo)

Date(s) of Playtesting: April 8, 2026

2. Purpose of the Player Test

To evaluate the game's playability, trap clarity, and overall fun factor before final submission. Specifically, to see if players understand the prank mechanics and whether the death delay improves feedback.

3. Participants

2 participants (both fellow university students, familiar with platformers but not the game).

Representative of target audience (casual gamers who enjoy humorous challenges).

4. Test Artifacts

Windows standalone build (Unity 2021.3).

Known limitations: no sound, one level, occasional camera clipping.

5. Tasks and Test Script

Directed tasks:

Move to the right using A/D and Space.

Find the hidden trap.

Trigger the falling platform.

Reach the door at the end.

Open-ended tasks: Explore freely for 10 minutes.

6. Metrics Collected

Number of deaths, time to first trap, completion time, and qualitative feedback via think-aloud.

7. Observations and Results

Both players died to the hidden trap on their first encounter. They laughed and said, "Oh, so that's the trick." The falling platform killed one player; the other jumped over it. Both completed the level. The death counter was noticed and appreciated.

8. Identified Problems

The falling platform sometimes does not kill if the player lands on the very edge.

No audio makes deaths feel less impactful.

9. Analysis and Interpretation

The edge collision issue is due to the platform's collider being slightly smaller than its sprite. This can be fixed by adjusting the collider size.

10. Proposed Design Changes

Increase the falling platform's collider size by 10%.

Add simple death sound (future).

Priority: Medium (not critical for demo).

11. Ethical Considerations

Participants were informed that the game contains cartoon violence and they could stop at any time. No personal data was recorded.

12. Reflection

The playtest confirmed that the death-delay feature successfully lets players see the trap. Future tests should include more participants and a structured questionnaire.

13. Conclusion

The game is fun and challenging; the core prank mechanics work as intended. Minor improvements are noted for future versions.